Task 2 – Explore Values and Types in the Interpreter

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 4 | Bloom's level: K4 – Analyse

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task02)

1. Interactive session

Python 3.12 interactive mode

```
>>> type(10)
<class 'int'>
>>> type(10.0)
<class 'float'>
>>> type("10")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type([10, 8.5])
<class 'list'>
>>> 7 / 2
3.5
>>> 7 // 2
3
>>> 7 % 2
1
>>> -7 // 2
-4
>>> -7 % 2
1
>>> 0.1 + 0.2
0.30000000000000004
>>> 0.1 + 0.2 == 0.3
False
>>> isinstance(True, int)
True
>>> "10" + "10"
'1010'
>>> 10 + float("10")
20.0
>>> from decimal import Decimal
>>> Decimal("0.1") + Decimal("0.2")
Decimal('0.3')
>>> Decimal(0.1)
Decimal('0.1000000000000000055511151231257827021181583404541015625')
>>> round(0.1 + 0.2, 2) == 0.3
True
>>> round(2.675, 2)
2.67
>>> round(0.5), round(1.5), round(2.5)
(0, 2, 2)
>>> int(19.99 * 100)
1998
>>> round(19.99 * 100)
1999
```

2. Explanation of each result

Expression	Result	Explanation
type(10), type(10.0), type("10"), type(True), type([10, 8.5])	int, float, str, bool, list	The value's form decides the type: 10.0 is a float because of the decimal point; quotes make a str; a list holds several values of any type.
7 / 2	3.5	**/** is true division and always returns a float.
7 // 2	3	**//** is floor division: the quotient rounded down.
7 % 2	1	Remainder, so that 7 = 2·3 + 1.
-7 // 2	-4	*Surprise:* floor division rounds toward minus infinity, not toward zero; −3.5 floors to −4. Then -7 % 2 is 1 so that 2·(−4) + 1 = −7 holds.

Expression	Result	Explanation
0.1 + 0.2	0.30000000000000004	*Surprise:* 0.1 and 0.2 have no exact binary form; the stored values are slightly off and the sum is a tiny bit more than 0.3.
0.1 + 0.2 == 0.3	False	The computer compares the stored binary numbers, not the decimals we typed. Decimal(0.1) shows the real stored value 0.1000000000000000055511...
isinstance(True, int)	True	*Surprise:* bool is a subclass of int (True is 1), so True + True is 2.
"10" + "10" vs 10 + float("10")	'1010' and 20.0	+ joins strings but adds numbers; this is why input text must be converted.
round(2.675, 2)	2.67	*Surprise:* 2.675 is stored as 2.67499999..., so it rounds down.
round(0.5), round(1.5), round(2.5)	0, 2, 2	*Surprise:* round() uses banker's rounding (ties go to the even number), not "half up".
int(19.99*100) vs round(19.99*100)	1998 vs 1999	19.99×100 is 1998.9999999999998; int() cuts it off, round() gives the intended 1999 paise.

3. Why money must not be added as float rupees

Float stores binary approximations, so each addition adds a tiny error and the errors accumulate; a bank total is exact to the paisa and the two will disagree. The alternatives are: **integer paise** (Rs 33.40 = 3340), **round()** after every operation, or the **decimal** module which stores decimal digits exactly.

4. Complete the Code – TODO 1

milk_money.py (TODO 1 replaced)

```
from decimal import Decimal
import sys, timeit

# 1,000 payments of Rs 33.40 and three small adjustments of Rs 0.10
payments = ["33.40"] * 1000 + ["0.10"] * 3


def total_float(items):
    total = 0.0
    for amount in items:
        total += float(amount)
    return total


def total_decimal(items):
    # TODO 1 completed: exact addition using Decimal built from each string
    total = Decimal("0")
    for amount in items:
        total += Decimal(amount)
    return total


print("float  :", total_float(payments))
print("Decimal:", total_decimal(payments))
print(0.1 + 0.2 == 0.3, Decimal("0.1") + Decimal("0.2") == Decimal("0.3"))
```

Output (identical to the expected output)

```
float  : 33400.30000000061
Decimal: 33400.30
False True
```

5. Change Decimal(amount) to Decimal(float(amount))

Output

```
Decimal(float(amount)) : 33400.29999999999857893118278
Decimal(float('33.40')): 33.399999999999985789145284797996282577514648437 5
```

The result becomes 33400.299999999998578931182 78 instead of 33400.30. `float("33.40")` is already the inexact binary number 33.39999999999999857...; Decimal then copies that error exactly. A Decimal is only exact when it is built from the **string** the clerk typed, never from a float.

Alternative Concept and Complexity Analysis

(a) Integer paise: convert each amount with `round(float(a)*100)`, add as int, divide by 100 only when printing. **(b) float + round():** `t = round(t + float(a), 2)` after every addition.

Measured (alternatives.py)

```
integer paise          : 3340030 paise  -> 33400.3 rupees
float + round()        : 33400.3
exact Decimal          : 33400.30

memory per value (sys.getsizeof):  int 28  float 24  Decimal 104
time per addition float (plain)       103 ns
time per addition integer paise       245 ns
time per addition float + round()     457 ns
time per addition Decimal             254 ns
```

Method	Time / addition	Memory / value	Exact bank total for n payments?
float (plain)	≈ 100 ns	24 bytes	No – 33400.30000000061 (error grows with n)
float + round()	≈ 460 ns	24 bytes	Gave 33400.3 here, but not guaranteed: round() on a float tie can go the wrong way (Task 10 shows 7,324 wrong paise)
integer paise	≈ 250 ns	28 bytes	Yes – integers are exact; needs care when converting from text
Decimal (from str)	≈ 250 ns	104 bytes	Yes – exact decimal arithmetic and explicit rounding modes

All are O(1) per addition and O(n) for n payments; the difference is only the constant factor and correctness. Decimal costs about 4× the memory of a float but is the only one that is both exact and readable.

Evidence

Annotated session with types, division operators and the 0.1 + 0.2 issue, resolved with Decimal built from strings. Source: Task02/milk_money.py, Task02/alternatives.py.